Suite Script 2.0 Client Script

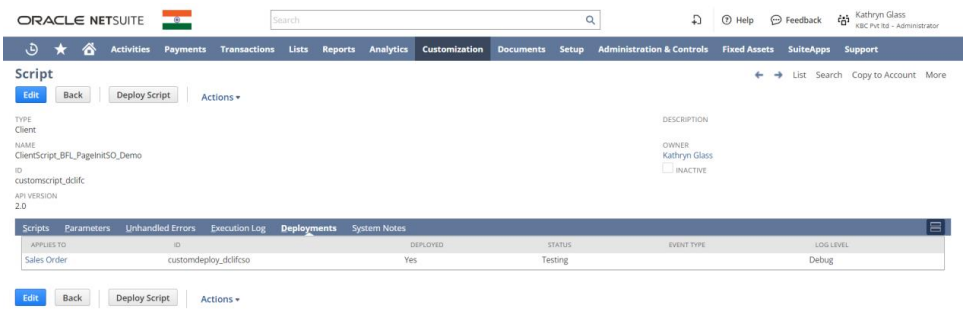
1. Upon loading the sales order, the script should set a customer in the customer field.

Ans.

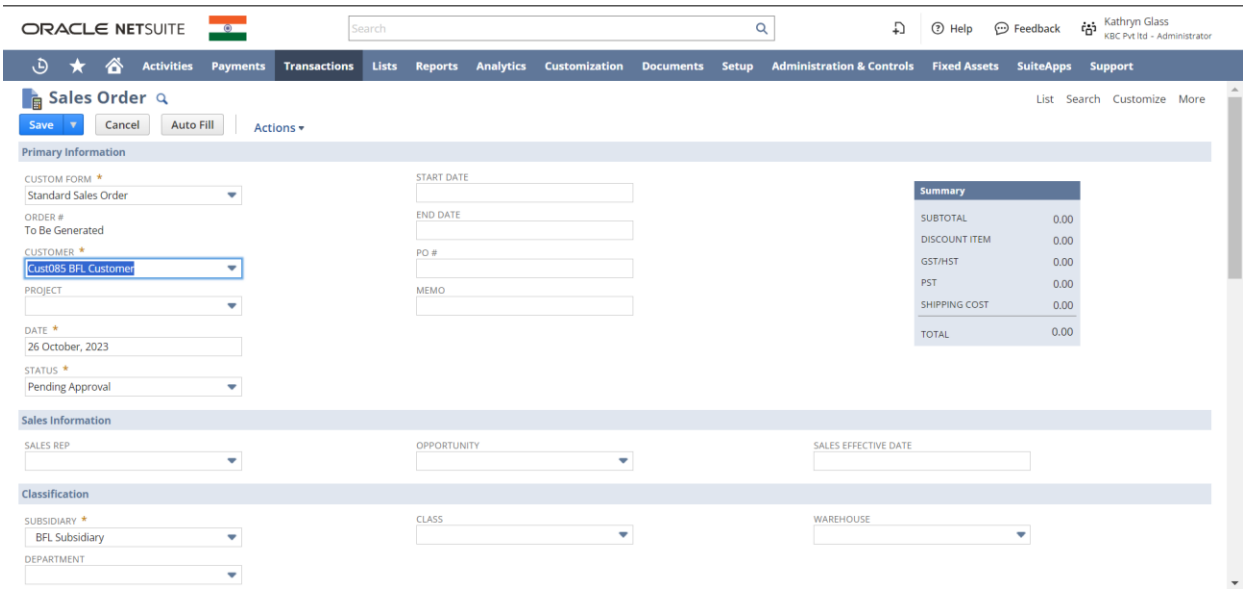
- In client script we can use pageInit event to autofill the customer in sales order.
- First, we will take current record object using currentRecord.get()
- Then we will check record type if it is sales order then we will set "BFL Customer" in customer field of sales order upon loading

```
/**
* @NApiVersion 2.x
* @NScriptType ClientScript
* @NModuleScope SameAccount
*/
define(['N/currentRecord'],

function(currentRecord) {
    var Record_Object;
    var Record_Type;
    function pageInit(scriptContext) {
      Record_Object = currentRecord.get();
      Record_Type = Record_Object.type;
        if(Record_Type == 'salesorder'){
            Record_Object.setValue({
            fieldId : 'entity',
            value: 1453,
        });
        }


    }

    return {
        pageInit : pageInit,
    };
});
```

- Script has deployed on sales order



- Customer name has set on sales order form



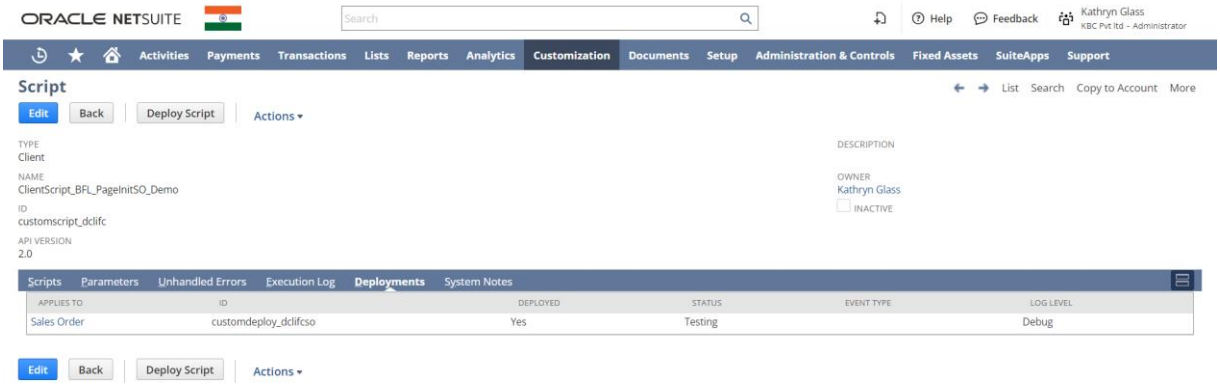
2. When the customer is changed, the script should retrieve the new customer's name and display it in the memo field.

Ans.

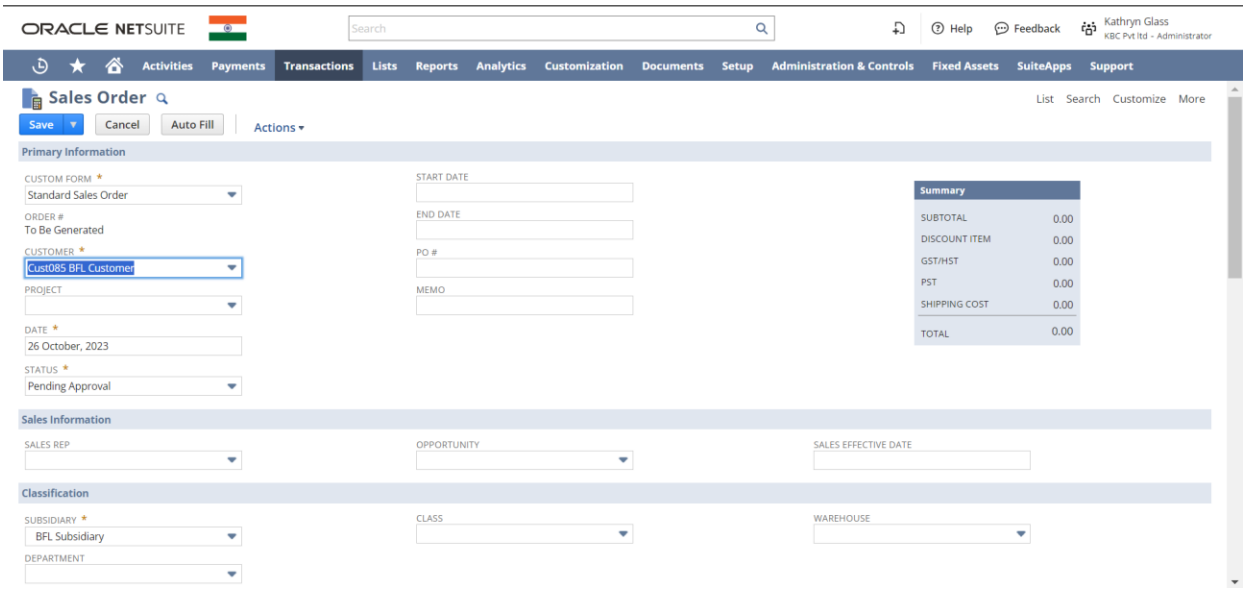
- In client script we can use fieldChange event to check is customer name change in sales order.
- First, we will get value of field using Record_Object.getText({fieldId: 'entity'});
- Then we will set customer name in memo field using Record_Object.setValue({fieldId: 'memo', value: value,});

```
define(['N/currentRecord'],

function(currentRecord) {
    var Record_Object;
    var Record_Type;
    function pageInit(scriptContext) {
      Record_Object = currentRecord.get();
      Record_Type = Record_Object.type;
        if(Record_Type == 'salesorder'){
            Record_Object.setValue({
            fieldId : 'entity',
            value: 1453,
        });
        }


    }
    function fieldChanged(scriptContext) {
        if(scriptContext.fieldId == 'entity'){
                var value = objRecord.getText({
                    fieldId: 'entity'
                });
                Record_Object.setValue({
                    fieldId : 'memo',
                    value: value,
                });
        }
    }

    return {
        pageInit : pageInit,
        fieldChanged: fieldChanged
    };
});
```

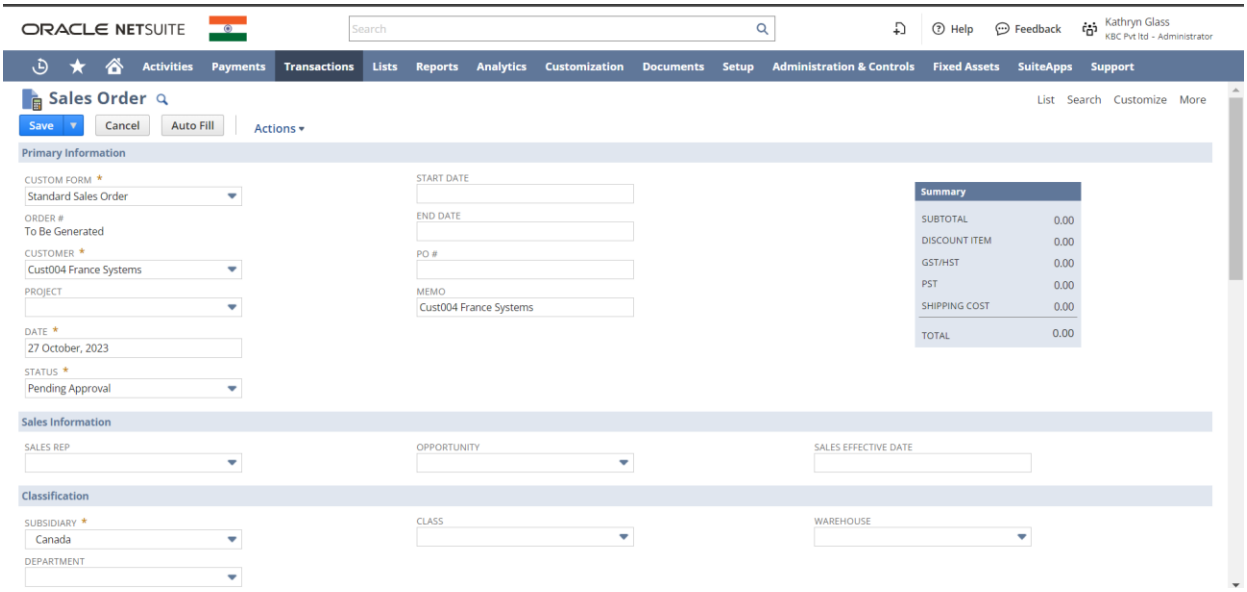
- Script has deployed on sales order



- Customer name has autofill on sales order form



- If we change customer name, then it will also show in memo field.



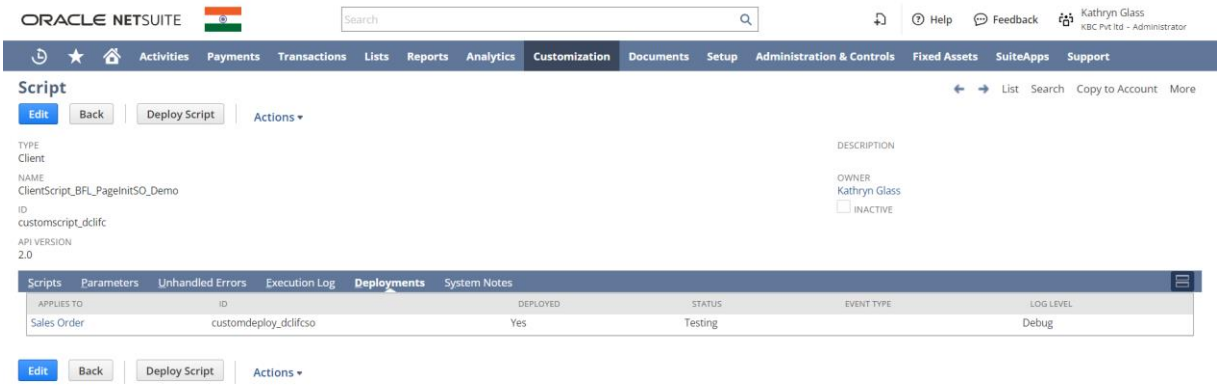
3. Do let the line be added if the quantity for the item is less than 3.

Ans.

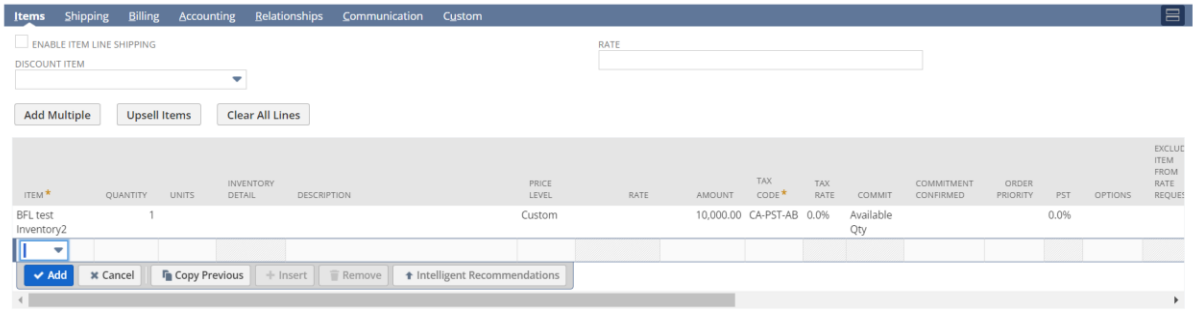
- In client script we can use validateLine event to validate quantity of items
- First, we will get value of quantity field using Record_Object.getCurrentSublistValue({sublistId: 'item', fieldId: 'quantity'});
- If it is less than 3 and more than 0 then add item else return false and alert.

```
function validateLine(scriptContext) {

  var quantity_value =  Record_Object.getCurrentSublistValue({
    sublistId: 'item',
    fieldId: 'quantity'
   });
   if(quantity_value > 0 && quantity_value < 3){
     return true;
   }
   else{
       alert("Quantity should be less than 3");
       return false;
   }



  }

  return {
      pageInit : pageInit,
      fieldChanged: fieldChanged,
      validateLine: validateLine
  };
});
```

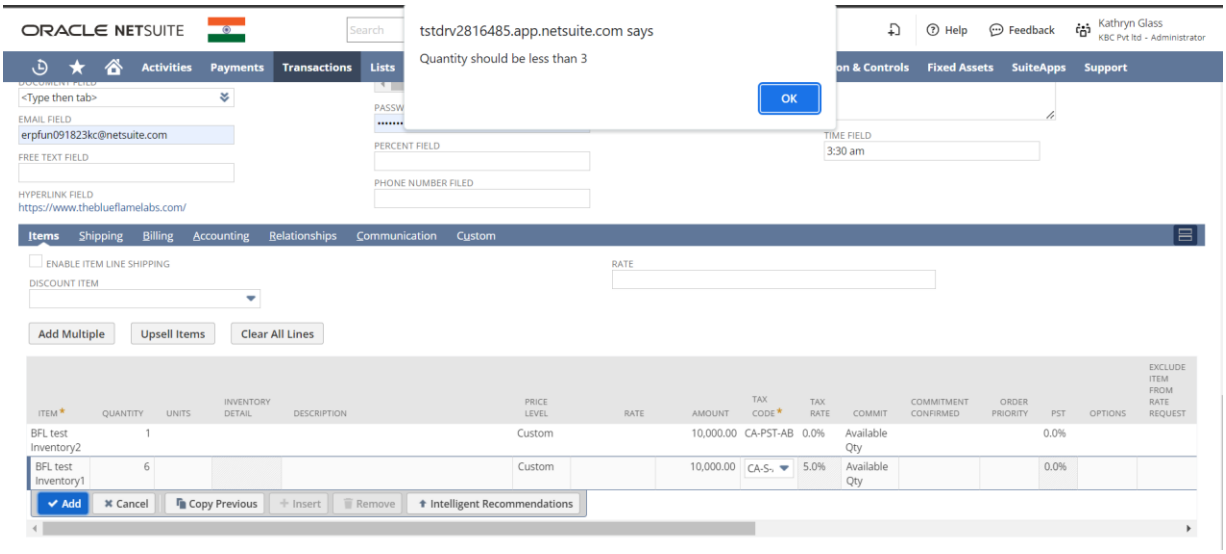
- Script has deployed on sales order



- Item with 1 quantity has been added to sales order



- Item with 6 quantities won't be added to sales order and will give alert.



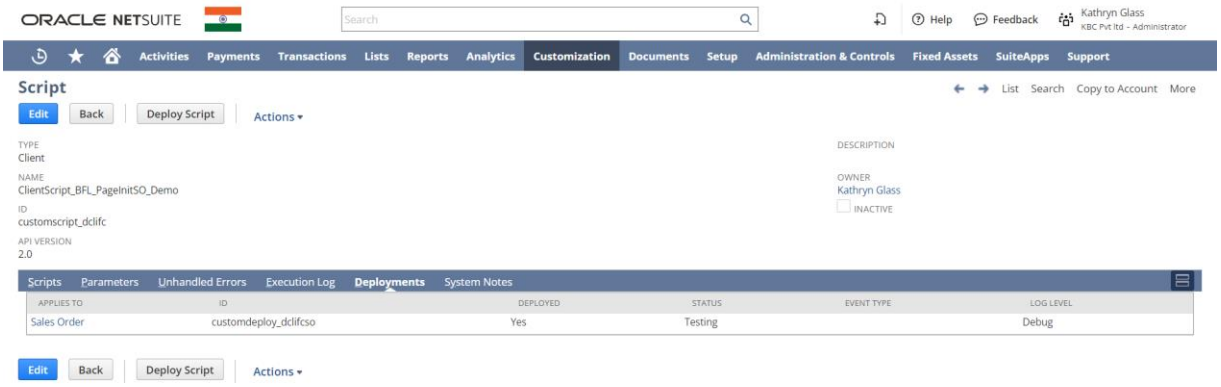
4. Do not line be deleted if the quantity for the item is 4.

Ans.

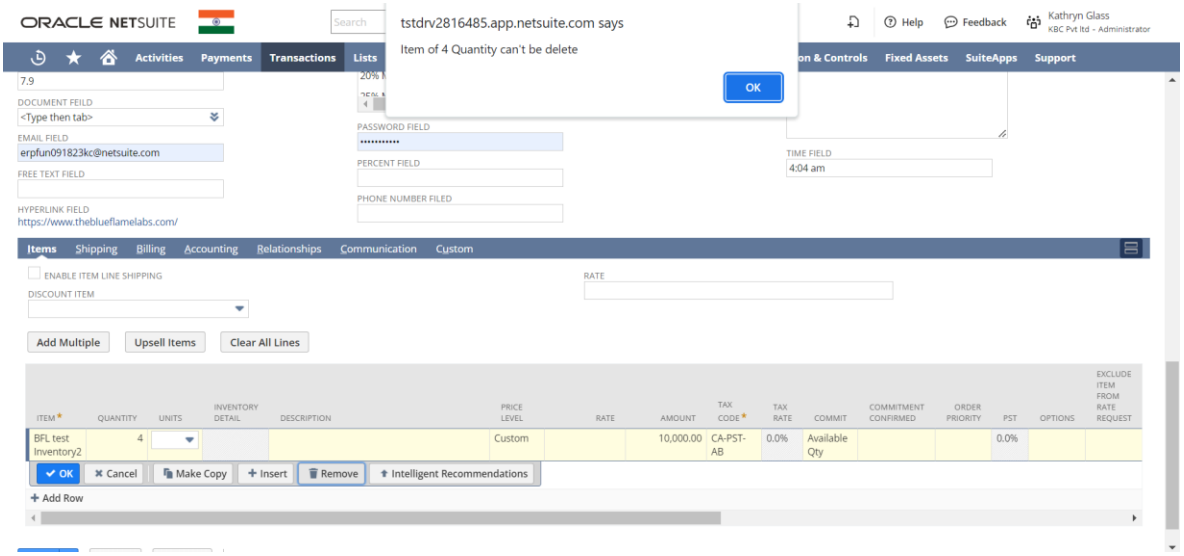
- In client script we can use validateDelete event to validate quantity of items
- First, we will get value of quantity field using Record_Object.getCurrentSublistValue({sublistId: 'item', fieldId: 'quantity'});
- If it is equal to 4 then return false and return alert

```
function validateDelete(scriptContext) {
quantity_value = Record_Object.getCurrentSublistValue({
  sublistId: "item",
  fieldId: "quantity",
});
if (quantity_value == 4) {
  alert("Item of 4 Quantity can't be delete");
  return false;
}

return true;
}


return {
    pageInit : pageInit,
    fieldChanged: fieldChanged,
    validateLine: validateLine,
    validateDelete: validateDelete
};
});
```

- Script has deployed on sales order



- Item with 4 quantities won't be deleted from sales order and will give alert.



5. Write all trigger points and show trigger name in alert function.

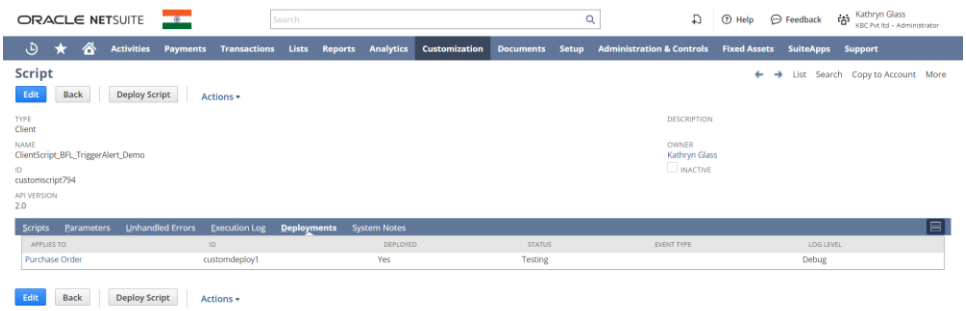
Ans.

- Create alert for all trigger events of Client script

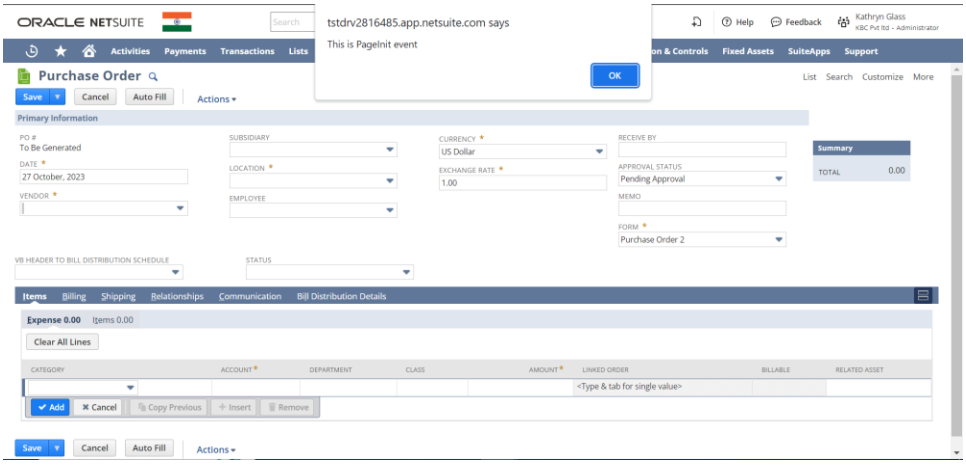
```
/**
 * @NApiVersion 2.x
 * @NScriptType ClientScript
 * @NModuleScope SameAccount
 */
define(['N/currentRecord'],

function(currentRecord) {

    function pageInit(scriptContext) {
        alert("This is PageInit event")
    }

    function fieldChanged(scriptContext) {
        if(scriptContext.fieldId == entity){
            alert("This is fieldChanged event")
        }
    }
    function postSourcing(scriptContext) {
        if(scriptContext.fieldId == entity){
            alert("This is postSoucing event")
        }
    }
    function sublistChanged(scriptContext) {
        alert("This is sublistChanged event")

    }

    function lineInit(scriptContext) {
        alert("This is lineInit event")

    }

    function validateField(scriptContext) {
        alert("This is validateField event")
    }
```

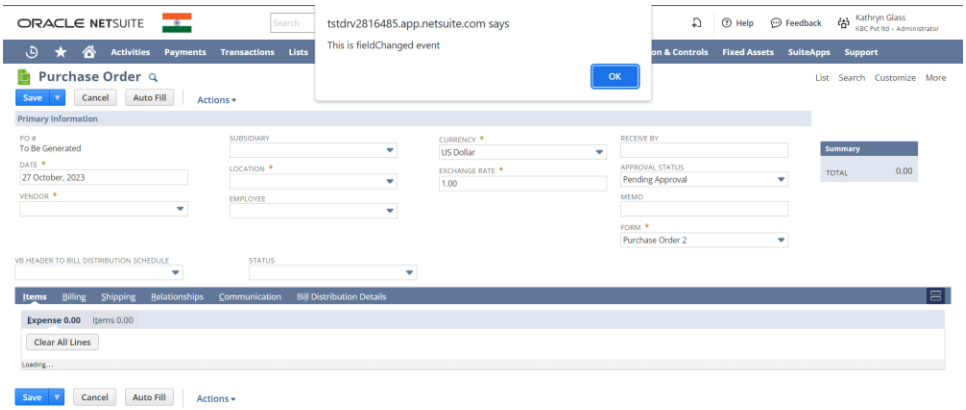
```
        alert("This is validateField event")
    }

    function validateLine(scriptContext) {
        alert("This is validateLine event")

    }

    function validateInsert(scriptContext) {
        alert("This is validateInsert event")
    }

    function validateDelete(scriptContext) {
        alert("This is validateDelete event")

    }

    function saveRecord(scriptContext) {
        alert("This is saveRecord event")
    }

    return {
        pageInit: pageInit,
        fieldChanged: fieldChanged,
        postSourcing: postSourcing,
        sublistChanged: sublistChanged,
        lineInit: lineInit,
        validateField: validateField,
        validateLine: validateLine,
        validateInsert: validateInsert,
        validateDelete: validateDelete,
        saveRecord: saveRecord
    };

});
```

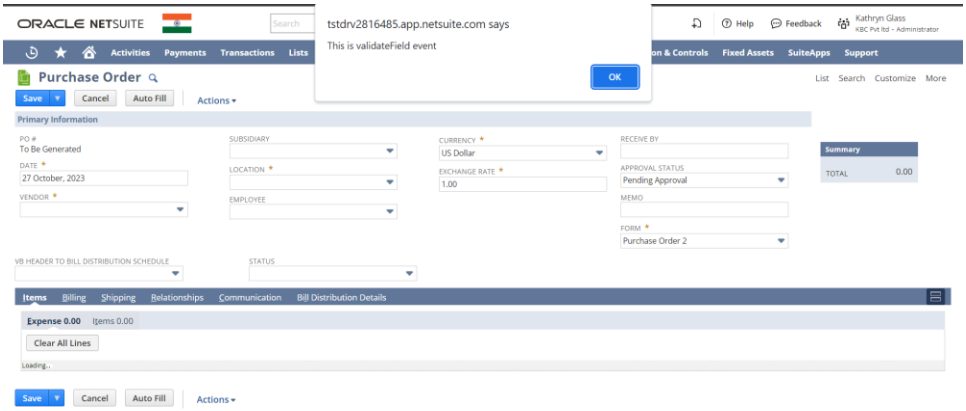
- Script has been deployed on purchase order.

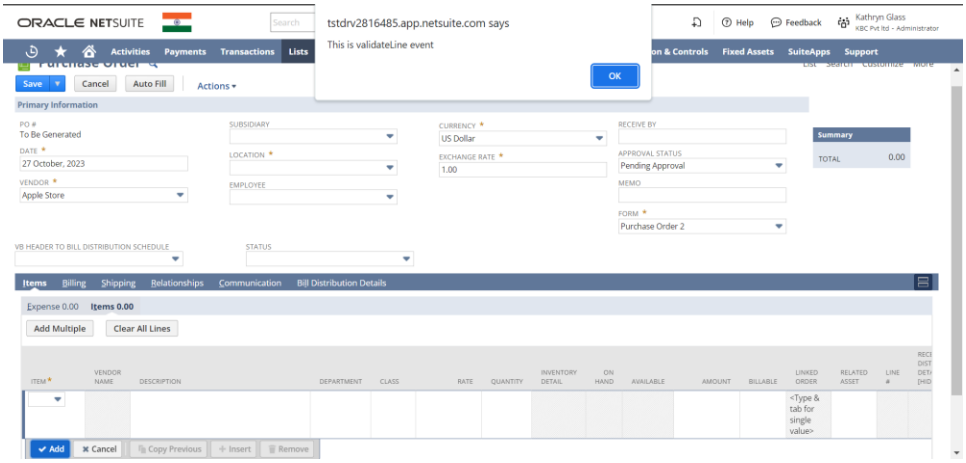


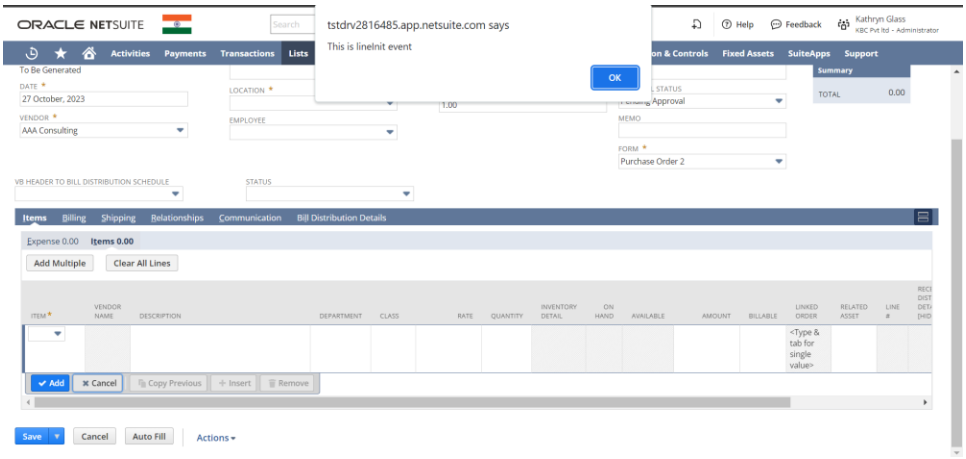
- Trigger Events

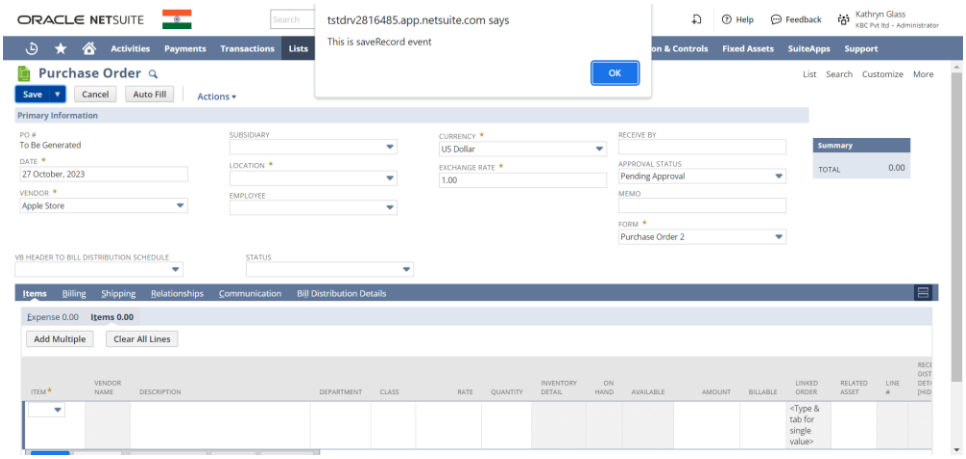



ORACLE NETSUITE
tstdrv2816485.app.netsuite.com says
This is fieldChanged event
OK
Help
Feedback
Kathryn Glass
KBC Pvt ltd - Administrator
Activities
Payments
Transactions
Lists
Fixed Assets
SuiteApps
Support
Purchase Order
List
Search
Customize
More
Save
Cancel
Auto Fill
Actions
Primary Information
PO #
To Be Generated
DATE *
27 October, 2023
VENDOR *
SUBSIDIARY
LOCATION *
EMPLOYEE
CURRENCY *
US Dollar
EXCHANGE RATE *
1.00
RECEIVE BY
APPROVAL STATUS
Pending Approval
MEMO
FORM *
Purchase Order 2
Summary
TOTAL
0.00
VB HEADER TO BILL DISTRIBUTION SCHEDULE
STATUS
Items
Billing
Shipping
Relationships
Communication
Bill Distribution Details
Expense 0.00
Items 0.00
Clear All Lines
Loading...


ORACLE NETSUITE
tstdrv2816485.app.netsuite.com says
This is validateField event
OK
Help
Feedback
Kathryn Glass
KBC Pvt ltd - Administrator
Activities
Payments
Transactions
Lists
Fixed Assets
SuiteApps
Support
Purchase Order
List
Search
Customize
More
Save
Cancel
Auto Fill
Actions
Primary Information
PO #
To Be Generated
DATE *
27 October, 2023
VENDOR *
SUBSIDIARY
LOCATION *
EMPLOYEE
CURRENCY *
US Dollar
EXCHANGE RATE *
1.00
RECEIVE BY
APPROVAL STATUS
Pending Approval
MEMO
FORM *
Purchase Order 2
Summary
TOTAL
0.00
VB HEADER TO BILL DISTRIBUTION SCHEDULE
STATUS
Items
Billing
Shipping
Relationships
Communication
Bill Distribution Details
Expense 0.00
Items 0.00
Clear All Lines
Loading..


ORACLE NETSUITE
tstdrv2816485.app.netsuite.com says
This is validateLine event
OK
Help
Feedback
Kathryn Glass
KBC Pvt ltd - Administrator
Activities
Payments
Transactions
Lists
Fixed Assets
SuiteApps
Support
Save
Cancel
Auto Fill
Actions
Primary Information
PO #
To Be Generated
DATE *
27 October, 2023
VENDOR *
Apple Store
SUBSIDIARY
LOCATION *
EMPLOYEE
CURRENCY *
US Dollar
EXCHANGE RATE *
1.00
RECEIVE BY
APPROVAL STATUS
Pending Approval
MEMO
FORM *
Purchase Order 2
Summary
TOTAL
0.00
VB HEADER TO BILL DISTRIBUTION SCHEDULE
STATUS
Items
Billing
Shipping
Relationships
Communication
Bill Distribution Details
Expense 0.00
Items 0.00
Add Multiple
Clear All Lines
ITEM *
VENDOR NAME
DESCRIPTION
DEPARTMENT
CLASS
RATE
QUANTITY
INVENTORY DETAIL
ON HAND
AVAILABLE
AMOUNT
BILLABLE
LINKED ORDER
RELATED ASSET
LINE #
<Type & tab for single value>
Add
Cancel
Copy Previous
Insert
Remove


ORACLE NETSUITE
tstdrv2816485.app.netsuite.com says
This is linelnit event
OK
Activities
Payments
Transactions
Lists
Help
Feedback
Kathryn Glass
KBC Pvt ltd - Administrator
Fixed Assets
SuiteApps
Support
To Be Generated
DATE *
27 October, 2023
VENDOR *
AAA Consulting
LOCATION *
EMPLOYEE
1.00
Approval
MEMO
FORM *
Purchase Order 2
Summary
TOTAL
0.00
VB HEADER TO BILL DISTRIBUTION SCHEDULE
STATUS
Items
Billing
Shipping
Relationships
Communication
Bill Distribution Details
Expense 0.00
Items 0.00
Add Multiple
Clear All Lines
ITEM *
VENDOR NAME
DESCRIPTION
DEPARTMENT
CLASS
RATE
QUANTITY
INVENTORY DETAIL
ON HAND
AVAILABLE
AMOUNT
BILLABLE
LINKED ORDER
RELATED ASSET
LINE #
<Type & tab for single value>
Add
Cancel
Copy Previous
Insert
Remove
Save
Cancel
Auto Fill
Actions


ORACLE NETSUITE
tstdrv2816485.app.netsuite.com says
This is saveRecord event
OK
Activities
Payments
Transactions
Lists
Help
Feedback
Kathryn Glass
KBC Pvt ltd - Administrator
Fixed Assets
SuiteApps
Support
Purchase Order
List
Search
Customize
More
Save
Cancel
Auto Fill
Actions
Primary Information
PO #
To Be Generated
DATE *
27 October, 2023
VENDOR *
Apple Store
SUBSIDIARY
LOCATION *
EMPLOYEE
CURRENCY *
US Dollar
EXCHANGE RATE *
1.00
RECEIVE BY
APPROVAL STATUS
Pending Approval
MEMO
FORM *
Purchase Order 2
Summary
TOTAL
0.00
VB HEADER TO BILL DISTRIBUTION SCHEDULE
STATUS
Items
Billing
Shipping
Relationships
Communication
Bill Distribution Details
Expense 0.00
Items 0.00
Add Multiple
Clear All Lines
ITEM *
VENDOR NAME
DESCRIPTION
DEPARTMENT
CLASS
RATE
QUANTITY
INVENTORY DETAIL
ON HAND
AVAILABLE
AMOUNT
BILLABLE
LINKED ORDER
RELATED ASSET
LINE #
<Type & tab for single value>

6. Difference between beforeLoad and pageInit.

Ans.

	beforeLoad	pageInit
1	beforeLoad triggered before the record form is loaded. This executes on the server-side before the form is presented to the user.	pageInit triggered after the record form is loaded. Executes on the client-side (browser) after the form is rendered, allowing for user interaction and interface modifications.
2	beforeLoad used for server-side processing, such as fetching data from related records, applying logic to set field values, or dynamically modifying the form structure. Basically, used for tasks that don't require user interaction.	pageInit used for client-side customization, enabling us to enhance the user experience by adding interactive features or dynamically updating fields based on user input such as validation, real-time calculations, or field-level events.
3	beforeLoad executes faster on the server before the form is presented to the user	pageInit might take latency or delays, if it involves client-side request & response action
4	This event has access to the server-side context, allowing us to work with record data and related records. We can access and manipulate record data before it's displayed to the user.	This event operates in the client-side context and has access to the browser's Document Object Model (DOM). We can use for tasks that involve modifying the user interface, handling client-side events, and reacting to user interactions

7. Difference between beforeSubmit and savedRecord.

Ans.

	beforeSubmit	savedRecord
1	beforeSubmit scripts execute before a record is saved to the database. They are typically used for data validation, manipulation, and field value adjustments before the record is saved.	savedRecord scripts execute after a record has been saved to the database and the user interface has been updated. This is primarily used for making changes to the user interface or performing client-side actions after the record is already saved.
2	This is used for server-side data validation, ensuring data integrity before saving. This can be used for business rules or conditions before allowing a record to be saved. This allows us to adjust field values, calculate values, or perform server-side data transformations.	This is used for enhancing the user experience and adding client-side interactivity. This is used for basic validation like mandatory filed or record type etc. But server-side complex validation won't be feasible with saveRecord.
3	This executes on the server side, which means it directly affects the data being saved to the database. This ensures data accuracy and consistency.	This executes on the client side (in the user's browser) and doesn't directly affect the data being saved. This can be only used for adding client-side functionality.
4	Probably beforeSubmit will trigger on fields and records	Probably savedRecord will trigger on save or submit button

8. What is sublistChanged, lineInit ,validateLine ,validateDelete.

Ans.

sublistChanged:

- This event handler is triggered when a change is made to a sublist. Sublists are a way to display related records within a primary record, in a tabular format.
- The sublistChanged event allows us to define custom behavior when changes are made to a sublist, such as adding or removing line items. We can use this event to perform validation or trigger other actions in response to sublist changes.

lineInit:

- This event handler is triggered when a new line (item) is added to a sublist. It allows us to set default values for fields on the new line or perform other actions when a new line is initialized.

validateLine:

- This event handler is used to validate data in a sublist line before it is saved. It is often used to enforce business rules and perform data validation on individual sublist lines.
- If the data does not meet the validation criteria, we can prevent the line from being saved and display an error message to the user.

validateDelete:

- This event handler is triggered when a user attempts to delete a line from a sublist. It allows us to validate whether the line can be deleted based on certain conditions or business rules.
- If the validation fails, we can prevent the deletion and provide alert to the user.

Order of Execution : beforeLoad -> pageInit -> saveRecord -> BeforeSubmit -> AfterSubmit